Playwright python Project Integration with Jenkins

How to Run a Local Playwright Project in Jenkins:

Step 1: Login to Jenkins

Open your Jenkins instance in a browser and log in using your credentials.

Step 2: Create a New Jenkins Job

- Click on **"New Item"**.
- Enter a name for your job.
- Select **"Freestyle project"**.
- Click **OK** to proceed.

Step 3: Configure the Job

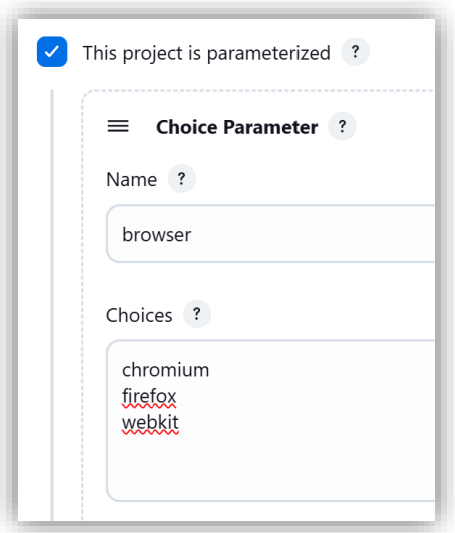
Enable Parameters

- Check the box **"This project is parameterized"**.
- Click **Add Parameter → Choice Parameter**.
  - **Name:** browser
  - **Choices:**

    chromium

    firefox

    webkit



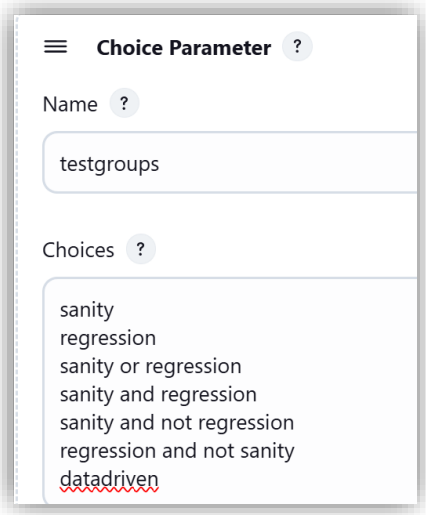
- Click **Add Parameter → Choice Parameter**.
    - **Name:** testgroups
    - **Choices:**

        sanity

        regression

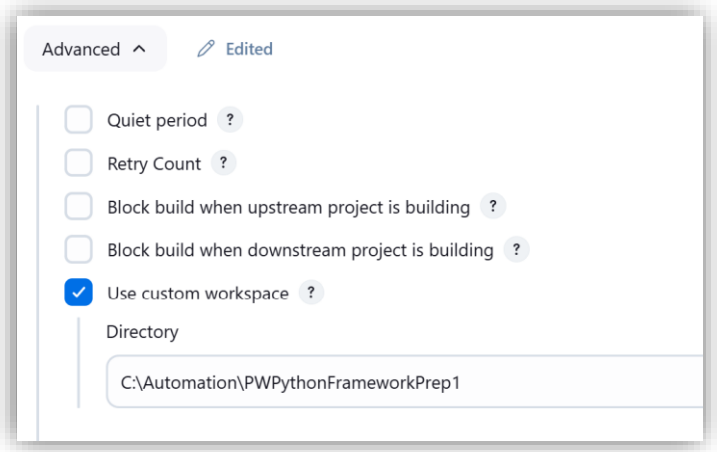
        sanity or regression

        sanity and regression



Set Custom Workspace

- Check **"Use custom workspace"**.
- Enter the full path to your local Playwright project directory.



Add Build Step

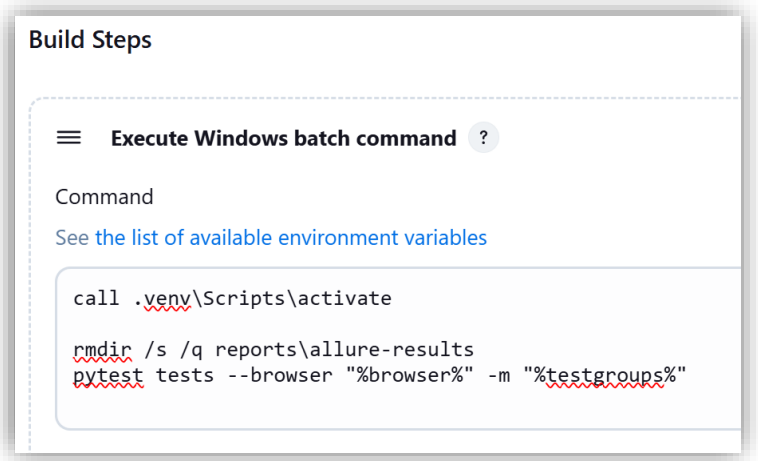
- Click on **"Add build step" → "Execute Windows batch command"** *(if on Windows)*.

- **Command:**

```
call .venv\Scripts\activate
rmdir /s /q reports\allure-results
pytest tests --browser "%browser%" -m "%testgroups%"
```

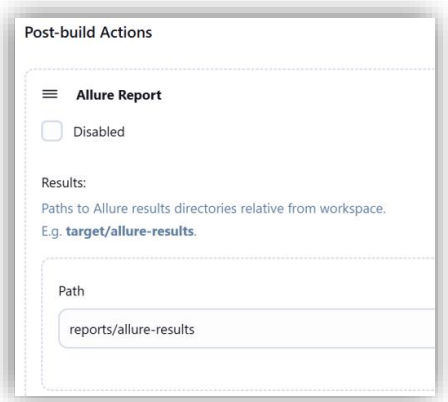
Note: If you're on **Mac/Linux**, select **"Execute shell"** and use:

```
call .venv\Scripts\activate
rmdir /s /q reports/allure-results
pytest tests\ --browser "$browser" -m "$testgroups"
```



Step 4: Post-build Actions (Allure Reports)

- Scroll to **"Post-build Actions"**.
- Select **"Allure Report"**.
- In **"Results path"**, enter:
- allure-results



Step 5: Save the Configuration

- Click **Apply** and then **Save** the job.

How to Run GitHub Playwright project in Jenkins

Step 1: Login to Jenkins

Open your Jenkins instance in a browser and log in using your credentials.

Step 2: Create a New Jenkins Job

- Click on **"New Item"**.
- Enter a name for your job.
- Select **"Freestyle project"**.
- Click **OK** to proceed.

Step 3: Configure the Job

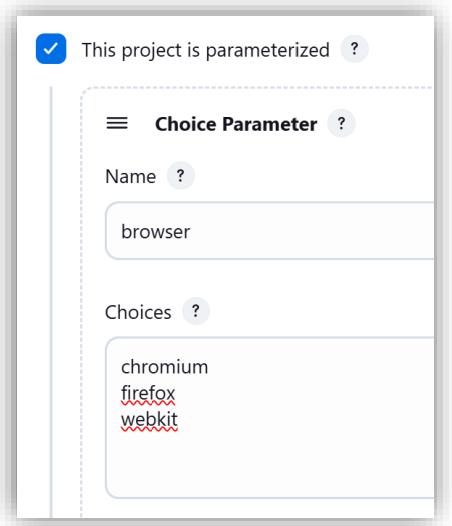
Enable Parameters

- Check the box **"This project is parameterized"**.
- Click **Add Parameter → Choice Parameter**.
  - **Name:** browser
  - **Choices:**

    chromium

    firefox

    webkit



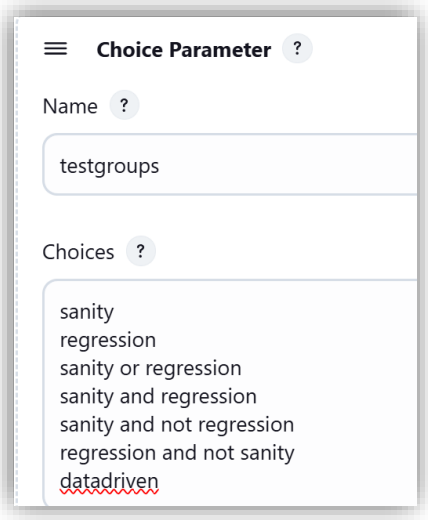
- Click **Add Parameter → Choice Parameter**.
  - **Name:** testgroups
  - **Choices:**

sanity

regression

sanity or regression

sanity and regression



Set GitHub Repository URL

- Go to '**Source Code Management'** section.
- Check **"Git"**.
- Provide GitHub repository URL.

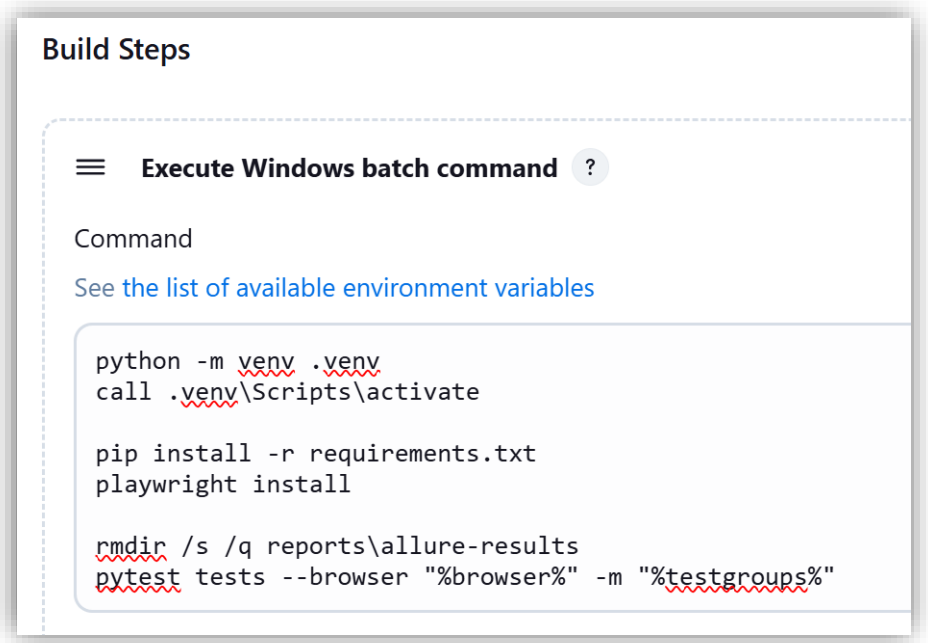
Add Build Step

- Click on **"Add build step" → "Execute Windows batch command"** *(if on Windows)*.
    - **Command:**

```
python -m venv .venv
call .venv\Scripts\activate

pip install -r requirements.txt
playwright install

rmdir /s /q reports\allure-results
pytest tests --browser "%browser%" -m "%testgroups%"
```

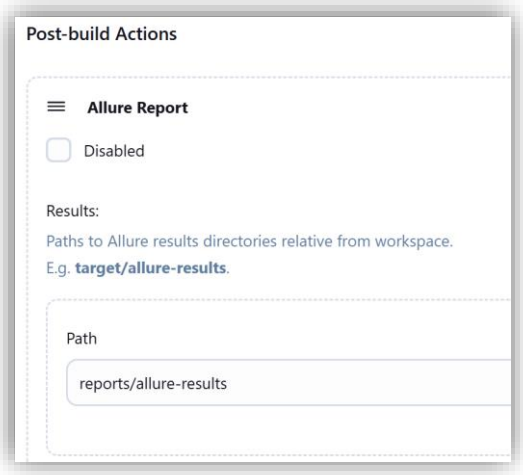
Note: If you're on **Mac/Linux**, select **"Execute shell"** and use:

```
python -m venv .venv
call .venv\Scripts\activate
pip install -r requirements.txt
playwright install

rmdir /s /q reports\allure-results
pytest tests\ --browser "$browser" -m "$testgroups"
```



Step 4: Post-build Actions (Allure Reports)

- Scroll to **"Post-build Actions"**.
- Select **"Allure Report"**.
- In **"Results path"**, enter:
- allure-results



Step 5: Save the Configuration

- Click **Apply** and then **Save** the job.

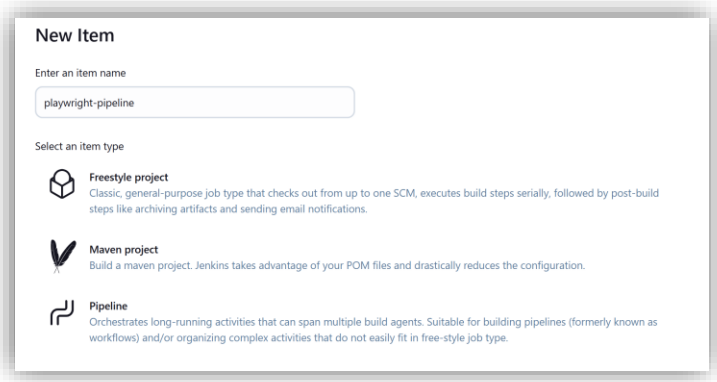
Run Github Playwright project in Jenkins with Pipeline

Step 1: Login to Jenkins

Open your Jenkins instance in a browser and log in using your credentials.

Step 2: Create a New Jenkins Job

- Click on **"New Item"**.
- Enter a name for your job.
- Select **"Pipeline project"**. (If you could not see this , then you need to install Pipeline plugin from manage Jenkins→plugins).
- Click **OK** to proceed.



Step 3: Configure the Job

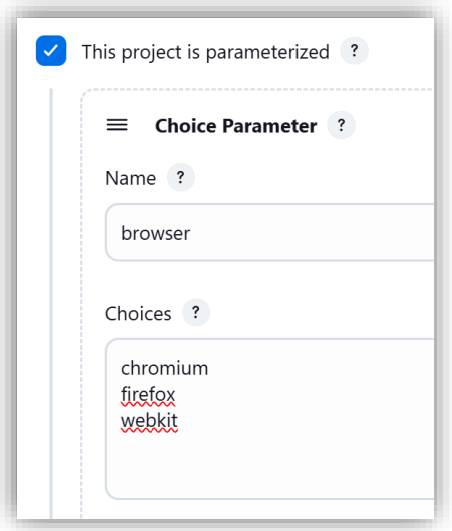
Enable Parameters

- Check the box **"This project is parameterized"**.
- Click **Add Parameter → Choice Parameter**.
  - **Name:** browser
  - **Choices:**

    chromium

    firefox

    webkit



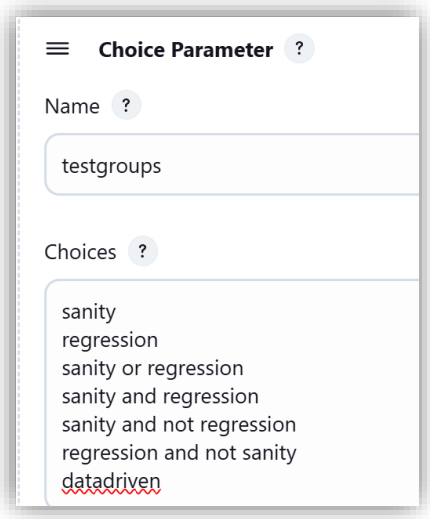
- Click **Add Parameter → Choice Parameter**.
  - **Name:** testgroups
  - **Choices:**

    sanity

    regression

    sanity or regression

    sanity and regression



Goto Pipeline section and add Pipeline script.

Pipeline Script (Windows):

```
pipeline {
    agent any


    environment {
        VENV_DIR = '.venv'
        REPORT_DIR = 'reports'
```

```
    }

    stages {

        // Step 1: Checkout Code from GitHub
        stage('Checkout Code') {
            steps {
                git 'https://github.com/pavanoltraining/PWPythonFrameworkPrep1.git'
                echo "Code checked out successfully."
            }
        }

        // Step 2: Setup Python Virtual Environment
        stage('Setup Python Environment') {
            steps {
                script {
                    if (!fileExists("${VENV_DIR}\\Scripts\\activate")) {
                        echo "Creating virtual environment..."
                        bat "python -m venv ${VENV_DIR}"
                    } else {
                        echo "Virtual environment already exists."
                    }
                }
            }
        }

        // Step 3: Install Required Packages
        stage('Install Dependencies') {
            steps {
                bat """
                    call ${VENV_DIR}\\Scripts\\activate
                    python -m pip install --upgrade pip
                    pip install -r requirements.txt
                    playwright install
                """
            }
        }

        // Step 4: Clean Old Test Reports
        stage('Clean Old Reports') {
            steps {
                bat """
                    if exist ${REPORT_DIR} (
                        rmdir /s /q ${REPORT_DIR}
                    )
                """
            }
        }

        // Step 5: Run Pytest Tests and Generate Allure Results
        stage('Run Tests') {
            steps {
                bat """
                    call ${VENV_DIR}\\Scripts\\activate
                    pytest tests --browser "%browser%" -m "%testgroups%" --
alluredir=${REPORT_DIR}\\allure-results
                """
            }
        }

        // Step 6: Publish Allure Report in Jenkins
        stage('Publish Allure Report') {
            steps {
                allure includeProperties: false, jdk: '', results: [[path:
"${REPORT_DIR}\\allure-results"]]
            }
        }
    }

    post {
```

```
        always {
            echo "Cleaning up workspace after build..."
        }
        success {
            echo "Tests executed successfully!"
        }
        failure {
            echo "Test execution failed!"
        }
    }
}
```

Pipeline Script (Max/Lnux):

```
pipeline {
    agent any

    // Optional parameters (you can remove if not needed)
    parameters {
        string(name: 'browser', defaultValue: 'chromium', description: 'Browser to run tests
on')
        string(name: 'testgroups', defaultValue: '', description: 'Test group or marker name')
    }

    environment {
        VENV_DIR = '.venv'
        REPORT_DIR = 'reports'
    }

    stages {

        // Step 1: Checkout Code from GitHub
        stage('Checkout Code') {
            steps {
                git 'https://github.com/pavanoltraining/PWPythonFrameworkPrep1.git'
                echo "Code checked out successfully."
            }
        }

        // Step 2: Setup Python Virtual Environment
        stage('Setup Python Environment') {
            steps {
                script {
                    if (!fileExists("${VENV_DIR}/bin/activate")) {
                        echo "Creating virtual environment..."
                        sh "python3 -m venv ${VENV_DIR}"
                    } else {
                        echo "Virtual environment already exists."
                    }
                }
            }
        }

        // Step 3: Install Required Packages
        stage('Install Dependencies') {
            steps {
                sh """
                    source ${VENV_DIR}/bin/activate
                    python3 -m pip install --upgrade pip
                    pip install -r requirements.txt
                    playwright install
                """
            }
        }

        // Step 4: Clean Old Test Reports
        stage('Clean Old Reports') {
            steps {
```

```
                sh """
                    if [ -d "${REPORT_DIR}" ]; then
                        rm -rf ${REPORT_DIR}
                    fi
                """
            }
        }

        // Step 5: Run Pytest Tests and Generate Allure Results
        stage('Run Tests') {
            steps {
                sh """
                    source ${VENV_DIR}/bin/activate
                    pytest tests --browser "${browser}" -m "${testgroups}" --
alluredir=${REPORT_DIR}/allure-results
                """
            }
        }

        // Step 6: Publish Allure Report in Jenkins
        stage('Publish Allure Report') {
            steps {
                allure includeProperties: false, jdk: '', results: [[path:
"${REPORT_DIR}/allure-results"]]
            }
        }
    }

    post {
        always {
            echo "Cleaning up workspace after build..."
        }
        success {
            echo "Tests executed successfully!"
        }
        failure {
            echo "Test execution failed!"
        }
    }
}
```

Step 4: Save the Configuration

- Click **Apply** and then **Save** the job.

Pipeline stages:

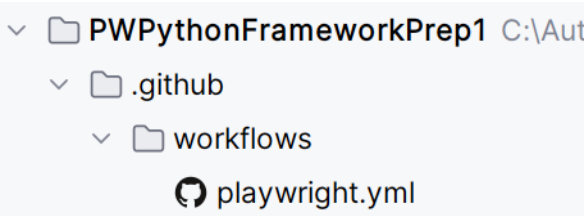
1. Checkout Code
2. Setup Python Environment
3. Install Dependencies
4. Clean Old Reports
5. Run Tests
6. Publish Allure Report

GitHub Actions:

On push/pull_request

Tests will run on push or pull request on branches main/master. The workflow will install all dependencies, install Playwright and then run the tests.

File location:



.github/workflows/playwright.yml

```
name: Playwright Tests
on:
  push:
    branches: [ master ]
  pull_request:
    branches: [ master ]
jobs:
  playwright:
    name: 'Playwright Tests'
    runs-on: ubuntu-latest
    container:
      image: mcr.microsoft.com/playwright/python:v1.55.0-noble
      options: --user 1001
    steps:
      - uses: actions/checkout@v5
      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: '3.13'
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Run your tests
        run: pytest
```